



United International University Department of CSE

Course Code: CSE 426/CSE 4326

Course Name: Microprocessor, Microcontroller and Interfacing Lab

Experiment no: 3

Experiment Name: : Introduction to Raspberry Pi (Gen 4 Model B/B+).

Submitted by:

Name: Md. Salman Kan

Student ID: 011 221 319

Section: H

Date of Performance: August 8, 2026

Date of Submission: August 9, 2026

Objective:

Raspberry Pi is defined as a minicomputer the size of a credit card that is interoperable with any input and output hardware device like a monitor, a television, a mouse, or a keyboard – effectively converting the set-up into a full-fledged PC at a low cost. Raspberry Pi is a programmable device. It comes with all the critical features of the motherboard in an average computer but without peripherals or internal storage. To set up the Raspberry computer, you will need an SD card inserted into the provided space. The SD card should have the operating system installed and is required for the computer to boot. Raspberry computers are compatible with Linux OS. This reduces the amount of memory needed and creates an environment for diversity.

Apparatus:

Raspberry Pi, MQ-series Gas Sensor, Jumper wires, Breadboard, Ultrasonic Sensor, LED, Resistor (220)

Theory:

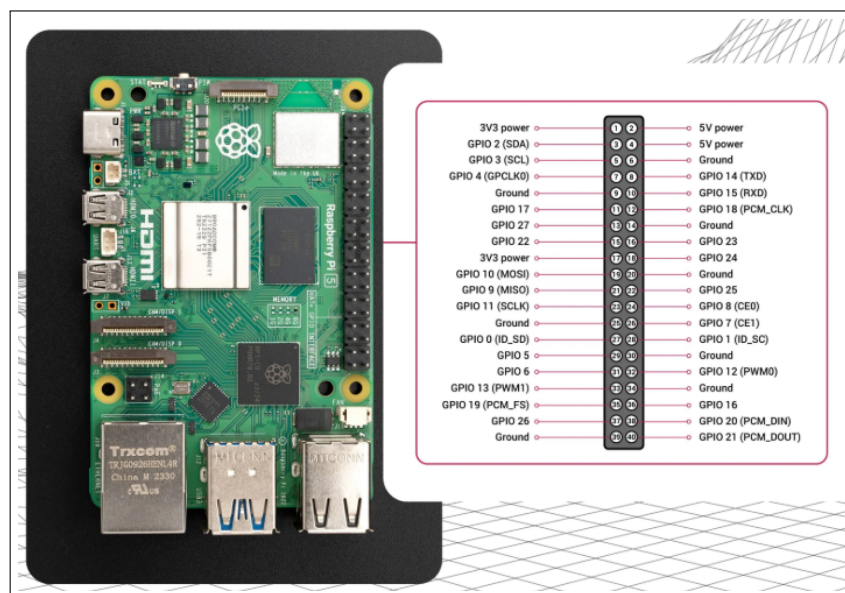


Figure 1: Raspberry Pi

Raspberry Pi is a family of small, affordable single-board computers created by the Raspberry Pi Foundation in the United Kingdom, first released in 2012 with the goal of making computing and programming accessible for education, hobbyists, and makers worldwide. These credit-card-sized devices feature a system-on-a-chip with ARM processors, onboard RAM, USB ports, HDMI output, GPIO pins for connecting sensors and

electronics, and support for microSD storage, enabling them to run full operating systems such as the official Raspberry Pi OS (a Debian-based Linux distribution) as well as other Linux variants, and in some cases Windows or specialized firmware. Over the years the lineup has expanded from the original Model B through successive generations including the Pi 2, 3, 4, and the current high-performance Raspberry Pi 5, alongside compact variants like the Raspberry Pi Zero series for ultra-low-cost projects and the Raspberry Pi Pico microcontroller boards based on the RP2040 chip for embedded applications. Typical uses range from teaching coding and electronics in schools, building media centers, retro gaming consoles, home automation systems, robotics, network servers, and Internet-of-Things devices, to industrial prototyping and scientific instrumentation, all while consuming very little power and costing only a fraction of a traditional desktop computer. The open hardware design, extensive community support, vast libraries of tutorials and software, and continuous hardware improvements have made Raspberry Pi one of the most popular platforms for learning computing, creative tinkering, and practical real-world projects across all ages and skill levels.

Solution:

1. Connect a gas sensor to the Raspberry Pi and answer the following:

- a) Write a python code to determine the gas volume in ppm data.
- b) Now, write a python code which will store that data in an excel file.

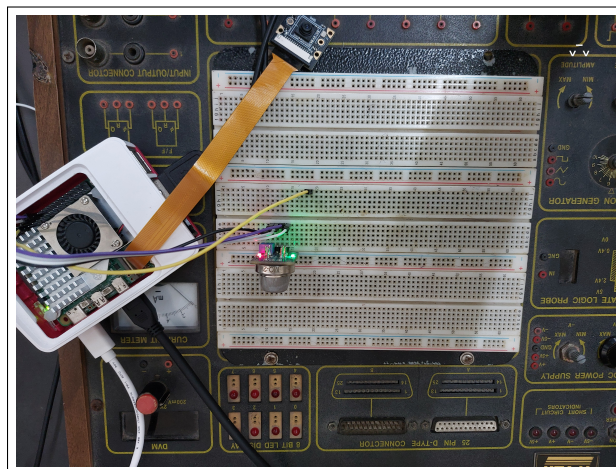


Figure 2: Gas Volume detection

```

1 from __future__ import annotations
2
3 import argparse
4 import csv
5 import os
6 import time
7 from datetime import datetime
8 from pathlib import Path
9
10 from gpiozero import Button
11
12 # MQ module digital output pin wired to Raspberry Pi GPIO.
13 GAS_DO_PIN = 27
14 READ_DELAY = 0.5
15 DEFAULT_LOG_FILE = Path(__file__).with_name("gas_digital_data.csv")
16 ESTIMATED_PPM_NORMAL = 25.0
17 ESTIMATED_PPM_GAS = 400.0
18
19 gas_input = Button(GAS_DO_PIN, pull_up=True)
20
21
22 def prepare_log_file(file_name: str):
23     """Create the CSV file with a header if it does not already
24     exist."""
25     if not os.path.exists(file_name):
26         with open(file_name, "w", newline="", encoding="utf-8")
27         as file_handle:
28             writer = csv.writer(file_handle)
29             writer.writerow(["Timestamp", "Gas State", "PPM"])
30
31 def read_gas_state() -> int:
32     """Read the digital gas alarm state from the MQ module.
33
34     Many MQ modules output 0 when gas is detected and 1 when the
35     level is
36     below the threshold. If your module behaves differently,
37     invert this logic.
38     """
39     return 0 if gas_input.is_pressed else 1

```

```

37
38
39 def estimate_ppm_from_state(gas_detected: bool) -> float:
40     """Return a rough ppm estimate from the digital state.
41
42     This is only a presentation aid for the D0 pin path. Because
43     the sensor
44     output is already thresholded, the result is not a real ppm
45     measurement.
46     """
47     return ESTIMATED_PPM_GAS if gas_detected else
48     ESTIMATED_PPM_NORMAL
49
50 def terminal_mode():
51     print("Reading MQ digital output. Press Ctrl+C to stop.\n")
52     print("0 usually means gas detected, 1 usually means normal
53     air.")
54     print("PPM values are only rough threshold-based values.\n")
55
56     try:
57         while True:
58             state = read_gas_state()
59             gas_detected = state == 0
60             estimated_ppm = estimate_ppm_from_state(gas_detected)
61
62             if gas_detected:
63                 print(f"Gas detected | PPM: {estimated_ppm:.2f}")
64             else:
65                 print(f"Normal air | PPM: {estimated_ppm:.2f}")
66
67             time.sleep(READ_DELAY)
68
69     except KeyboardInterrupt:
70         print("\nStopped by user.")
71     finally:
72         gas_input.close()
73
74 # EXCEL/CSV LOGGING CODE
75 def excel_mode(file_name: str):

```

```

74     prepare_log_file(file_name)
75     print(f"Logging digital gas state into {file_name}. Press
Ctrl+C to stop.\n")
76
77     try:
78         while True:
79             state = read_gas_state()
80             gas_detected = state == 0
81             estimated_ppm = estimate_ppm_from_state(gas_detected)
82
83             timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%
S")
84             status = "Gas detected" if gas_detected else "Normal
air"
85
86             with open(file_name, "a", newline="", encoding="utf-8
") as file_handle:
87                 writer = csv.writer(file_handle)
88                 writer.writerow([
89                     timestamp,
90                     status,
91                     f"{estimated_ppm:.2f}"
92                 ])
93
94                 print(
95                     f"{timestamp} | {status} | "
96                     f"PPM: {estimated_ppm:.2f} saved to CSV"
97                 )
98
99                 time.sleep(READ_DELAY)
100
101     except KeyboardInterrupt:
102         print("\nStopped by user.")
103     finally:
104         gas_input.close()
105
106
107 def build_parser():
108     parser = argparse.ArgumentParser(
109         description="MQ gas sensor without ADC"
110     )

```

```

111
112     subparsers = parser.add_subparsers(
113         dest="command",
114         required=True
115     )
116
117     subparsers.add_parser(
118         "gas-terminal",
119         help="Print gas detection state in terminal"
120     )
121
122     # EXCEL/CSV COMMAND
123     excel_parser = subparsers.add_parser(
124         "gas-excel",
125         help="Log gas detection state to CSV"
126     )
127
128     excel_parser.add_argument(
129         "--file",
130         default=str(DEFAULT_LOG_FILE),
131         help="Output CSV file name"
132     )
133
134     return parser
135
136
137 def main():
138     parser = build_parser()
139     args = parser.parse_args()
140
141     if args.command == "gas-terminal":
142         terminal_mode()
143     elif args.command == "gas-excel":
144         excel_mode(args.file)
145
146
147 if __name__ == "__main__":
148     main()

```

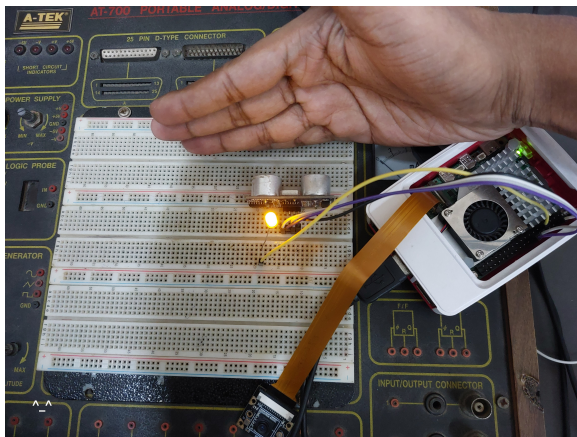
Output

Google Sheets Dataset.

Table 1: MQ Sensor Module Connections with Raspberry Pi

Device	Pin	Raspberry Pi Pin
MQ sensor module	VCC	5V
MQ sensor module	GND	GND
MQ sensor module	D0 / DO	GPIO27

2. Add an Ultrasonic Sensor to your RPi and a LED. Write a python code that will determine the distance of any object from the Ultrasonic sensor and print it in the terminal. Also, if the object is very close to the sensor, the LED should light up



(a) Object close to sensor

```

micro:~$ python3 -pi lab_codes.py ultrasonic
/usr/lib/python3/dist-packages/gpiozero/input_devices.py:852: PWMSoftwareFallback: For more accurate readings, use the pigpio pin factory
See https://gpiozero.readthedocs.io/en/stable/api_input.html#distancesensor-hc-sr04 for more info
WARNING: using PWMSoftwareFallback!
Measuring distance. Press Ctrl+C to stop.

Distance: 60.40 cm
Distance: 60.49 cm
Distance: 60.84 cm
Distance: 60.43 cm
Distance: 5.82 cm
Distance: 2.59 cm
Distance: 2.07 cm
Distance: 1.81 cm
Distance: 2.12 cm
Distance: 1.76 cm
Distance: 1.71 cm
Distance: 1.05 cm
Distance: 2.82 cm
Distance: 2.05 cm
Distance: 2.07 cm
Distance: 1.05 cm
Distance: 2.07 cm
Distance: 1.05 cm
Distance: 1.71 cm
Distance: 1.86 cm
Distance: 2.96 cm
Distance: 2.18 cm
Distance: 8.89 cm
Distance: 9.86 cm
Distance: 9.76 cm
Distance: 3.94 cm
Distance: 2.78 cm
Distance: 3.27 cm
Distance: 2.68 cm
Distance: 2.79 cm
Distance: 60.41 cm
Distance: 60.43 cm
Distance: 60.41 cm
^C
Stopped by user.
micro:~$

```

(b) Distance printed on terminal

Figure 3: Determine the distance of any object from the Ultrasonic sensor

Table 2: HC-SR04 Ultrasonic Sensor and LED Connections with Raspberry Pi

Device	Pin	Raspberry Pi Pin
HC-SR04	VCC	5V
HC-SR04	GND	GND
HC-SR04	TRIG	GPIO23
HC-SR04	ECHO	GPIO24
LED	Anode	GPIO17
LED	Cathode	GND

```

1 from gpiozero import DistanceSensor, LED
2 from time import sleep
3
4 # Pin configuration
5 TRIG = 23
6 ECHO = 24
7 LED_PIN = 17
8
9 # Set up ultrasonic sensor and LED
10 sensor = DistanceSensor(
11     echo=ECHO,
12     trigger=TRIG,
13     max_distance=2.0
14 )
15
16 led = LED(LED_PIN)
17
18 # Distance threshold for turning on the LED
19 CLOSE_DISTANCE = 10.0 # cm
20
21 print("Measuring distance. Press Ctrl+C to stop.\n")
22
23 try:
24     while True:
25         # Read distance in centimeters
26         distance = sensor.distance * 100
27
28         # Print distance in terminal
29         print(f"Distance: {distance:.2f} cm")
30
31         # Turn LED ON when object is very close
32         if distance <= CLOSE_DISTANCE:
33             led.on()
34         else:
35             led.off()
36
37         sleep(0.5)
38
39 except KeyboardInterrupt:
40     print("\nStopped by user.")
41

```

```
42 finally:
43     led.off()
44     sensor.close()
45     led.close()
```

Discussion:

The Pi was asking for a password to open SPI protocols and was unable to connect to the internet for a while. Later installed os once again, and things went better.